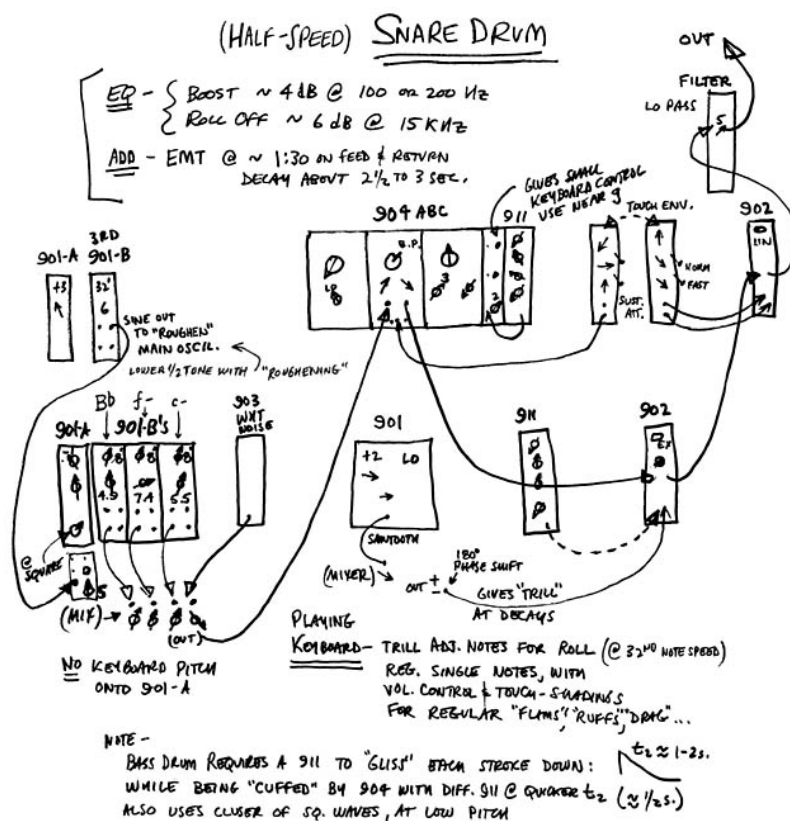


Getting Started with Max/MSP gen~

Introduction

gen~ is an environment that lives inside the visual programming language called **Max**. Instead of writing code line-by-line, in Max and gen~, we are patching virtual cables into "objects", similar to working with modular synthesizers. Just like other creative coding languages have special names for their files (for example, sketches for Processing or cartridges for PICO-8), Max files are called **patches**, like synth patches.



Patch notes drawn by Wendy Carlos in 1977

Sample by Sample

Within Max, you can generate and process images, sounds, live data, and more; however, **gen~** is specifically designed for working with sound at a sample-by-sample level. Normally, **MSP** objects are used for working with sound, as indicated by the tilde (~) symbol in their name. However, they process sound data streams in chunks of samples at a time ("signal vectors", "buffers", or "blocks").

Export gen~ patches as C++ code

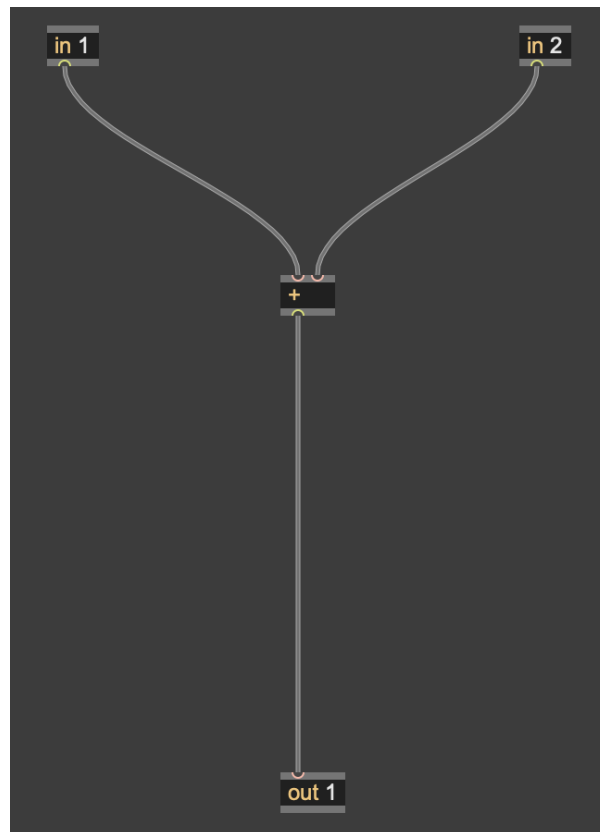
Because gen~ operates at such a low level, it makes it possible to export a patch as C++ code for use in other digital signal processing (DSP) applications. One example would be if you wanted to export your patch to a small microcontroller that specializes in sound for a site-specific installation or for creating a custom, portable instrument.

The gen~ object

While Max MSP and gen~ workflows are similar, they can be thought of as different languages. Open up a max patch and press the "n" shortcut to create a new object, type in gen~, and press Enter.



Double-clicking this object will take you to the gen~ environment. You can think of this environment as the subatomic world of digital sound.



The default patch of a newly create gen~ object.

With the ability to operate on single samples, we can start to explore new possibilities.

Familiarize yourself with the UI

Operators

The nature of sonic signal processing is a combination of signal streams and **operations**.

Operators can be found on the left sidebar, or you can type it when you press the **n** keyboard shortcut (for **new** operator). These gen~ operators might have identical names to Max MSP objects, but they are more low-level.



Ramps, The Universal Primitive of Repeating Change

Envelopes

Think of envelopes as windows. They contain the “activity” of a never-ending signal.

Things to Know

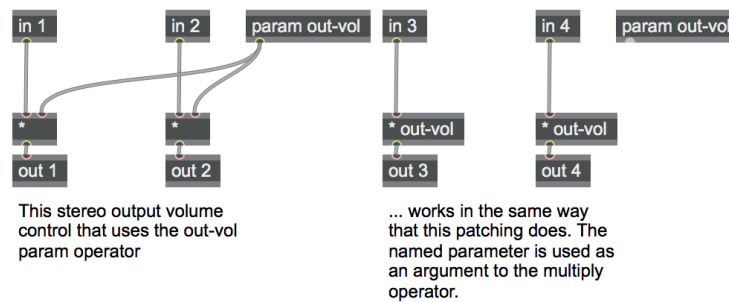
- All of the processing inside of a gen~ object resembles MSP because processing in gen~ is also **synchronous**.
- There are no cold or hot inlets.
- There is no right-to-left ordering when you think of output from gen~ operators.
- Objects in gen are “on” all the time. Even a float operator. The output from a gen~ object is always a signal – it’ll be a 32-bit or 64-bit signal, depending on which version of Max you’re running.



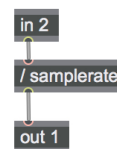
- Everything inside of the gen~ patching environment is done using 64-bit floating point numbers – in fact, you don't need to distinguish

between integer values and floating point numbers at all inside of your gen~ patcher (you'll probably notice that in the example patches and in the work posted to the Forum).

- When you use an argument to set a default value, that value set by the argument is fixed and cannot be changed.
- In Gen you can name parameter operators using an argument, and then use that parameter name when performing calculations. Using them can make your Gen patching a lot nicer looking.



Gen includes a number of useful and commonly used numbers you can use in patching



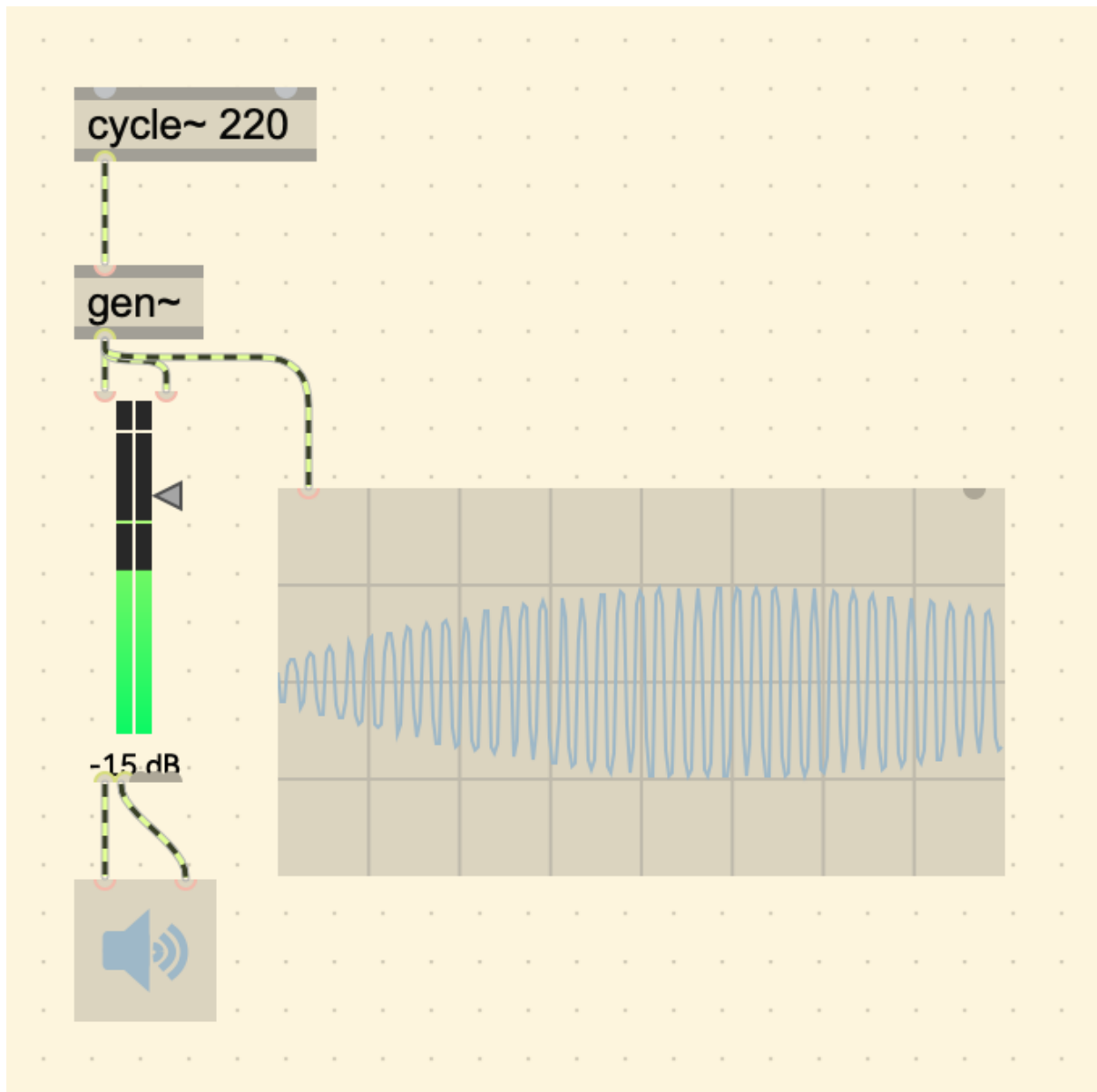
The gen~ object knows that your current samplerate is, and lets you use that value for calculations

1.618034 42

Do you need a constant value? Just type a number into an empty operator box!

- Constants
- In the gen~ world, a pulse is the equivalent of a bang message, a transition from a zero value to any non-zero value that happens within the time frame of a single sample.

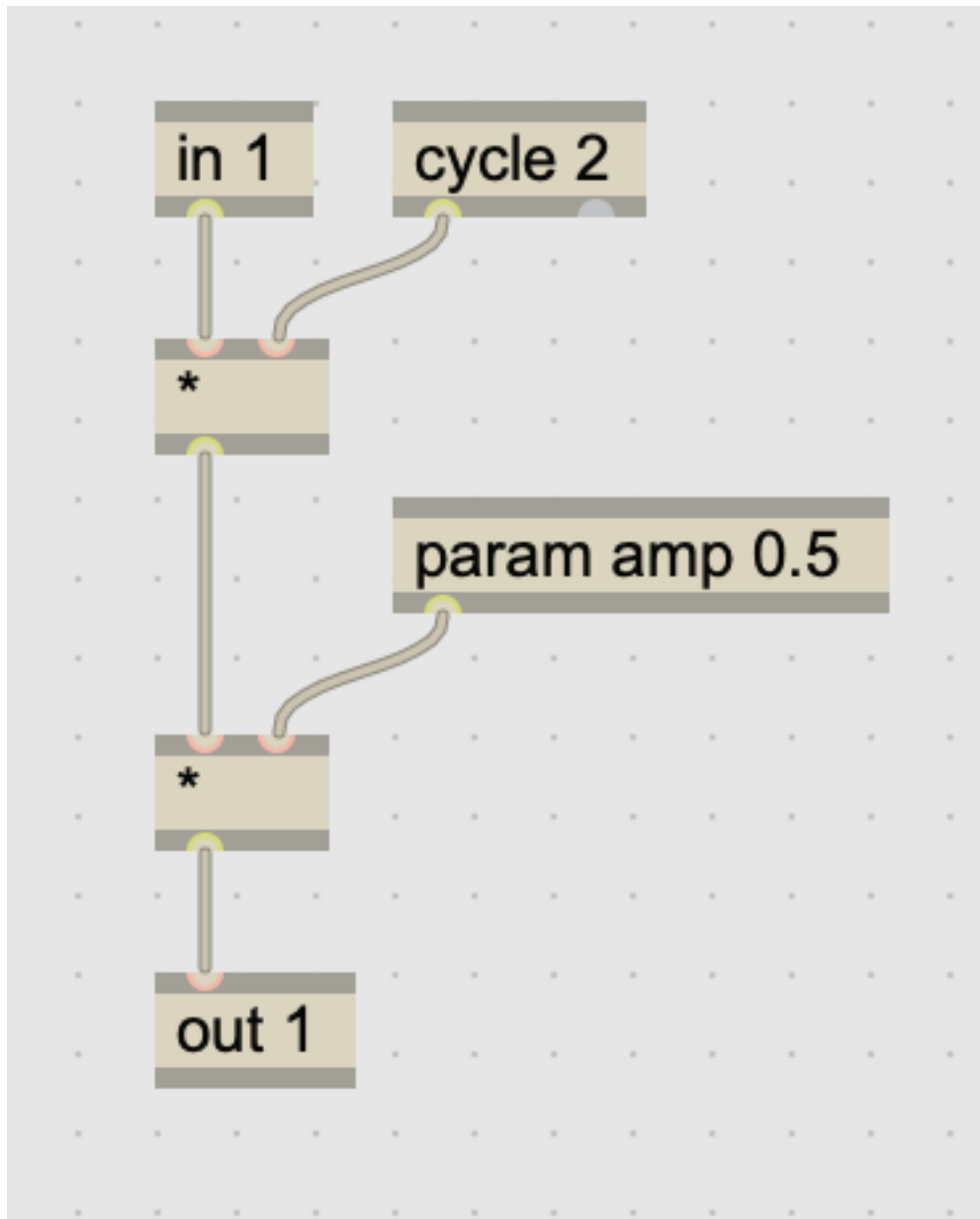
Basic Example



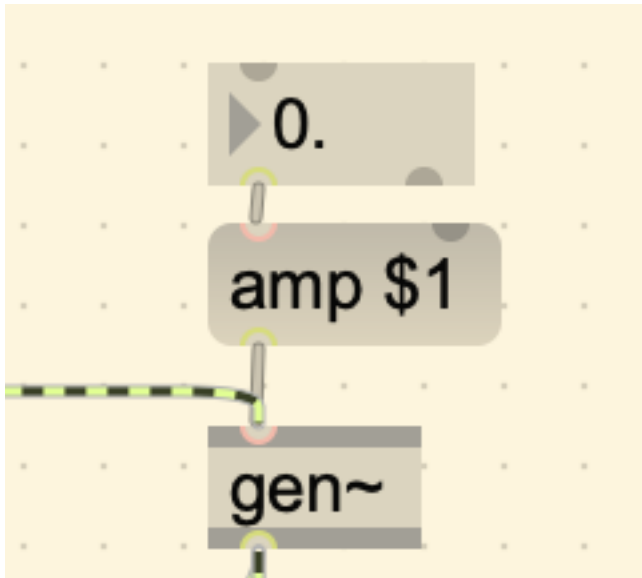
Here you have a normal max cycle~ object with a frequency of 220Hz going into a gen~ object. Inside the gen~ object we have this:

It defines that there is one input with the [in 1] object. The audio rate input that is coming in is then modulated by a gen~ [cycle] object with a frequency of 2Hz. The result of that is then attenuated by a default amplitude of 0.5, but can be defined as a parameter outside the gen~ environment. The result is the output of the [gen~] object.

You can enter the parameter for the amplitude by sending a message with the parameter name followed by its value.



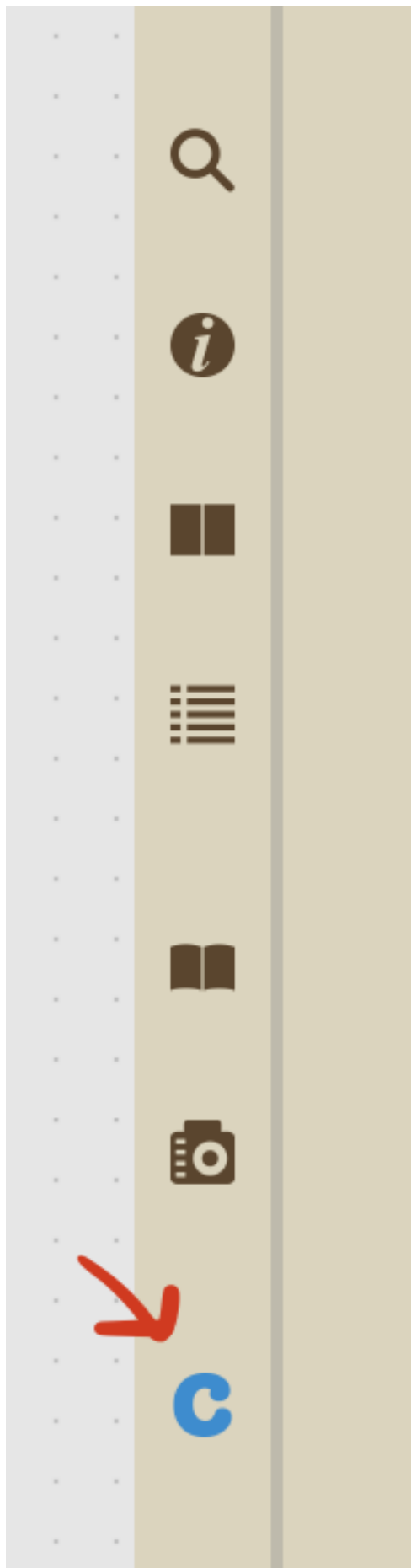
l400



You can set a min and max value for a parameter by setting the corresponding attributes.



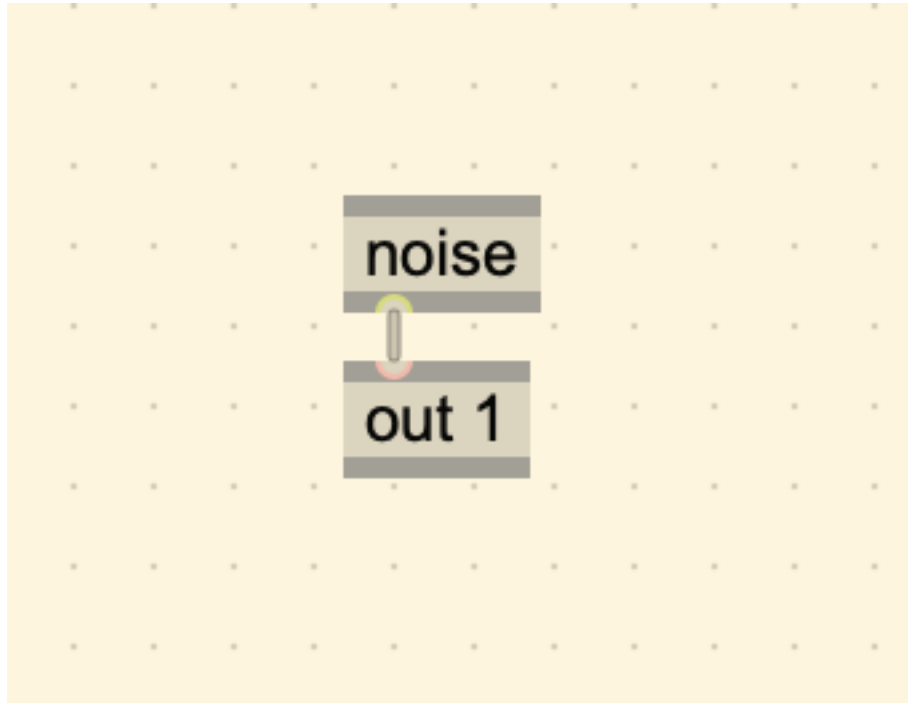
As you are making your patch, the equivalent text based code is being updated. To see it click on the 'C' icon on the right sidebar.



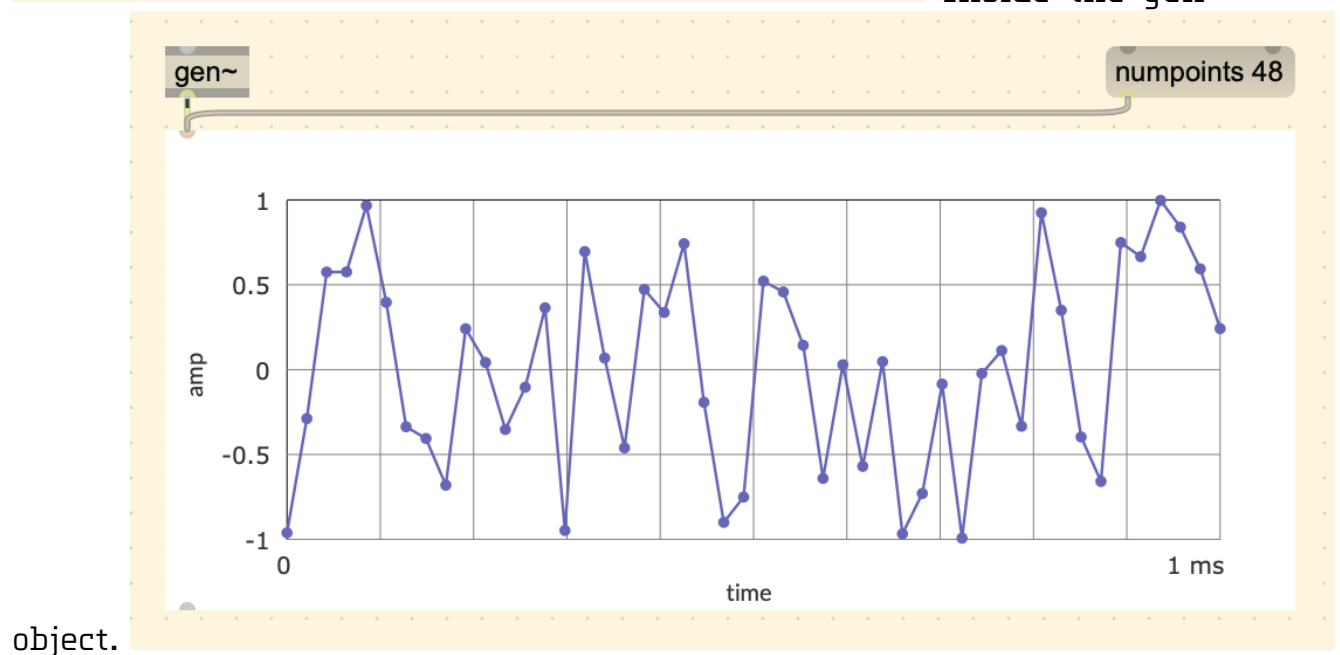
```
1 Param amp(0.5, max=1., min=0.);
2 cycle_1, cycleindex_2 = cycle;
3 mul_3 = in1 * cycle_1;
4 mul_4 = mul_3 * amp;
5 out1 = mul_4;
```

[noise]

It is just a random number generator. It outputs a random number between -1 and 1 every sample.



Inside the gen~



object.

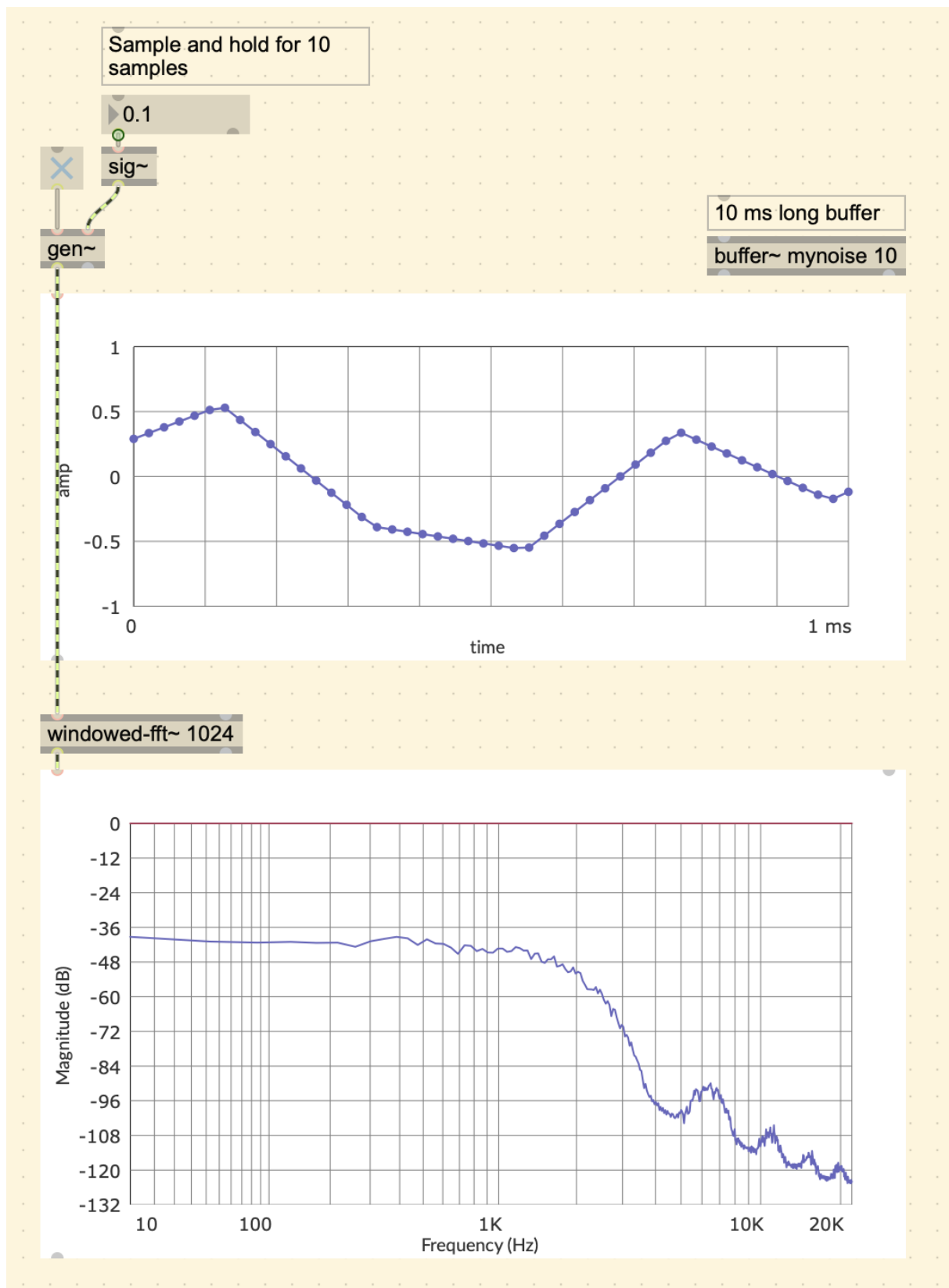
```
"data" : [ -0.962023636649588, -0.288658083802691, 0.574758381195038, 0.57
```

Data taken from the plot. Notice that they are 64 bit float values.



The `gen~` code writes a random value into the `mynoise` buffer every sample at the audio rate (48000 times a second in this case). At the same time, another counter is using `peek` to output the value of the buffer. In this second buffer, I can set the amount to add after each count, for example, instead of going through the buffer at the normal speed of 1 sample each time, I can go through it 10, 100, or 1000 samples each time. In this case, I set a number between 0 and 1, because there is no such thing as a decimal sample index, it will hold the same sample it is on until it crosses into another sample index. Notice how this removes high frequencies.

Now look at what happens when we interpolate linearly between these 10



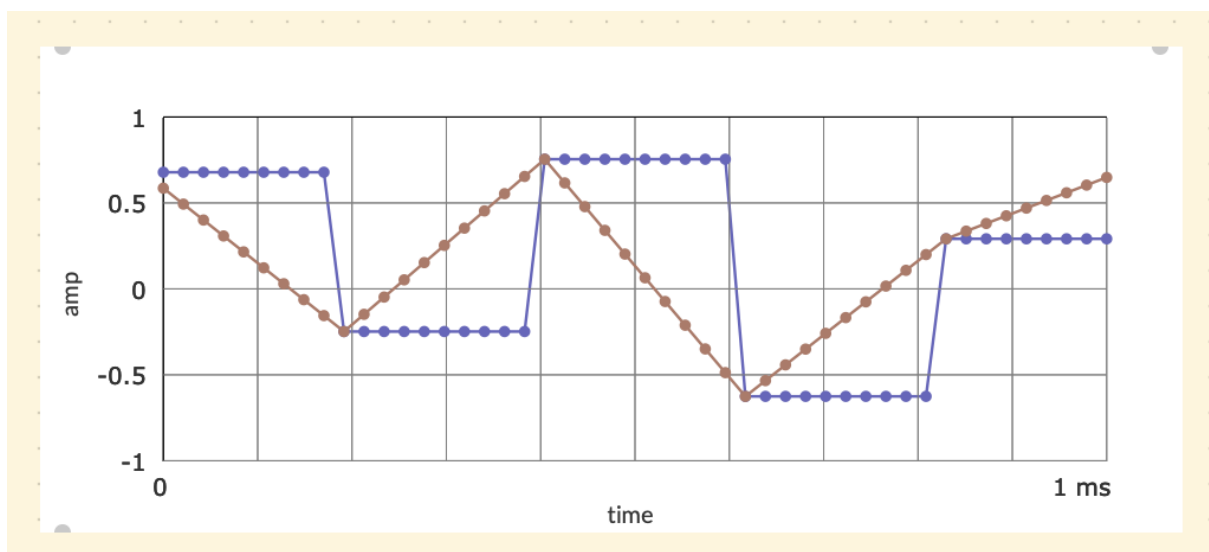
samples.



You

can interpolate by setting this attribute value.

Notice how even more high frequencies are removed.



Here we can clearly see what interpolation does. The blue is the sample and hold, and the red is with linear interpolation.

Here is the normal MSP noise for comparison, they are the same thing. A random number generator that outputs a float between -1 and 1.

noise~

